

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN MÔN HỌC
CƠ SỞ DỮ LIỆU PHÂN TÁN

ĐỀ TÀI 6: CHUYỂN TIỀN NGÂN HÀNG BẰNG
GIAO THỨC TWO-PHASE COMMIT (2PC)

Sinh viên:	Nguyen Phuoc Sang
MSSV:	
Lớp:	
Giảng viên hướng dẫn:	

Mục lục

1	Phần mở đầu	4
1.1	Tên đề tài	4
1.2	Phạm vi và đối tượng nghiên cứu	4
1.2.1	Đối tượng nghiên cứu	4
1.2.2	Phạm vi nghiên cứu	4
1.3	Mục tiêu	5
1.3.1	Mục tiêu tổng quát	5
1.3.2	Mục tiêu cụ thể	5
1.4	Phạm vi hệ thống	6
1.4.1	Phạm vi thực hiện	6
1.4.2	Ngoài phạm vi	6
1.5	Hệ thống đang ở mức demo hay production?	6
2	Cơ sở lý thuyết	7
2.1	CSDL phân tán là gì	7
2.2	Phân mảnh dữ liệu	7
2.2.1	Horizontal Fragmentation	7
2.2.2	Vertical Fragmentation	8
2.2.3	Hybrid Fragmentation	8
2.3	Replication	8
2.3.1	Snapshot Replication	8
2.3.2	Transactional Replication	8
2.4	Giao dịch phân tán	8
2.4.1	ACID	8
2.4.2	Two-Phase Commit (2PC)	9
3	Phân tích hệ thống	10
3.1	Phát biểu bài toán	10
3.2	Mô hình nghiệp vụ tổng quát	11
3.3	Use case	11

3.3.1	Tác nhân	11
3.3.2	Danh sách use case	12
3.4	Lược đồ quan hệ	12
4	Thiết kế CSDL phân tán	13
4.1	Phân tích Sites	13
4.2	Chiến lược phân mảnh	13
4.2.1	Horizontal fragmentation	13
4.2.2	Vertical fragmentation (có chủ đích)	14
4.2.3	Hybrid fragmentation	14
4.3	Giải thích lựa chọn thiết kế	14
4.3.1	Tại sao phân theo chi nhánh/site ngân hàng?	14
4.3.2	Tại sao không replicate full toàn bộ accounts?	14
4.4	Sơ đồ phân tán (site giữ bảng nào)	14
4.5	Chiến lược đồng bộ dữ liệu	15
4.5.1	Đồng bộ nghiệp vụ tiền tệ	15
4.5.2	Xử lý conflict	15
5	Cài đặt và cấu hình	16
5.1	Công nghệ sử dụng	16
5.2	Cấu hình kết nối phân tán	16
5.2.1	Database	16
5.2.2	Coordinator backend	16
5.2.3	Toxiproxy để mô phỏng lỗi mạng	16
5.3	Sơ đồ kết nối giữa các server	17
5.4	Liên hệ với template SQL Server/Linked Server/Replication	17
6	Xây dựng và Demo	18
6.1	Store Procedure phân tán (tương đương trong project hiện tại)	18
6.2	Distributed Transaction 2PC	19
6.2.1	Lệnh XA chính được sử dụng	19
6.2.2	Các endpoint demo chính	19
6.3	Test case	20
6.3.1	Bảng test case tổng hợp	20
6.3.2	Kết quả tự động hóa kiểm thử	20
6.3.3	Node chết / network delay / rollback scenario	21
6.4	Giao diện (screenshot)	21
7	Đánh giá hệ thống	22
7.1	Ưu điểm	22

7.2	Nhược điểm	22
7.3	So sánh với hệ tập trung	22
8	Kết luận và hướng phát triển	24
8.1	Tổng kết	24
8.2	Hướng phát triển	24
9	Tài liệu tham khảo	25
A	Phụ lục A - Trích đoạn API quan trọng	27
B	Phụ lục B - Ma trận test và CI	28

Chương 1

Phần mở đầu

1.1 Tên đề tài

Đề tài 6 - Chuyển tiền Ngân hàng: Two-Phase Commit (2PC).

Đề tài mô phỏng bài toán chuyển tiền liên ngân hàng trong hệ thống phân tán, yêu cầu giao dịch phải đảm bảo tính nguyên tử:

- Hoặc trừ tiền ở tài khoản nguồn và cộng tiền ở tài khoản đích đều thành công.
- Hoặc toàn bộ giao dịch bị hủy, không để xảy ra tình trạng lệch dữ liệu.

1.2 Phạm vi và đối tượng nghiên cứu

1.2.1 Đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là hệ thống cơ sở dữ liệu phân tán trong lĩnh vực ngân hàng, cụ thể là bài toán chuyển tiền giữa các tài khoản thuộc các chi nhánh khác nhau. Hệ thống được xây dựng nhằm mô phỏng hoạt động của các ngân hàng có nhiều site (chi nhánh) hoạt động độc lập nhưng vẫn đảm bảo tính nhất quán dữ liệu trên toàn hệ thống.

Bên cạnh đó, đề tài tập trung nghiên cứu giao thức Two-Phase Commit (2PC) để xử lý các giao dịch phân tán, đảm bảo các tính chất ACID, đặc biệt là tính nguyên tử (Atomicity) trong quá trình chuyển tiền giữa các site.

1.2.2 Phạm vi nghiên cứu

Phạm vi của đề tài được giới hạn trong các nội dung sau:

- Mô phỏng hệ thống gồm từ 2 đến 3 site ngân hàng, mỗi site quản lý một tập dữ liệu riêng biệt.
- Xây dựng chức năng chuyển tiền giữa các tài khoản thuộc các site khác nhau thông qua giao dịch phân tán.
- Áp dụng giao thức Two-Phase Commit (2PC) để đảm bảo tính toàn vẹn và nhất quán dữ liệu.
- Sử dụng hệ quản trị cơ sở dữ liệu SQL Server kết hợp với cơ chế Linked Server để kết nối giữa các site.
- Triển khai các stored procedure để thực hiện truy vấn và giao dịch phân tán.

Ngoài ra, đề tài không tập trung vào các vấn đề sau:

- Không triển khai trên môi trường thực tế với quy mô lớn (production system).
- Không xử lý các nghiệp vụ ngân hàng phức tạp như tín dụng, đầu tư hoặc quản lý rủi ro.
- Không tối ưu hiệu năng ở mức cao hoặc xử lý đồng thời số lượng lớn giao dịch.

1.3 Mục tiêu

1.3.1 Mục tiêu tổng quát

Xây dựng hệ thống chuyển tiền liên ngân hàng có khả năng chịu lỗi, sử dụng giao thức Two-Phase Commit trên nhiều CSDL độc lập.

1.3.2 Mục tiêu cụ thể

- Triển khai được giao dịch phân tán giữa ít nhất 2 site ngân hàng.
- Hỗ trợ các tình huống lỗi thường gặp: fail ở Phase 1, fail ở Phase 2, timeout, coordinator crash, participant crash.
- Có cơ chế *recovery* từ log để xử lý giao dịch treo.
- Có cơ chế *idempotency* để tránh double-submit.
- Có bộ test matrix TC01-TC16 và CI để tự động hóa kiểm thử.

1.4 Phạm vi hệ thống

1.4.1 Phạm vi thực hiện

- Backend: Flask (Python), API REST.
- Database: 3 MySQL site (bank1, bank2, bank3) chạy qua Docker Compose.
- Giao thức giao dịch phân tán: XA transaction + 2PC do coordinator điều phối.
- Frontend: HTML/CSS/JavaScript với màn hình monitor giao dịch.
- Mô phỏng lỗi mạng: Toxiproxy (cấu hình qua cổng 8666).

1.4.2 Ngoài phạm vi

- Chưa triển khai xác thực/phân quyền mức production (JWT/OAuth).
- Chưa triển khai replication cấp CSDL để HA thực sự.
- Chưa tối ưu cho tải cao quy mô lớn như production banking.

1.5 Hệ thống đang ở mức demo hay production?

Hệ thống trong đề tài này ở mức demo học thuật nâng cao (production-like):

- Đạt được: có đầy đủ flow 2PC, recovery, compensation, idempotency, monitor, CI test.
- Chưa đạt production đầy đủ: chưa có cluster HA thực sự, chưa hardening security, chưa có observability và disaster recovery cấp doanh nghiệp.

Chương 2

Cơ sở lý thuyết

2.1 CSDL phân tán là gì

CSDL phân tán là hệ thống dữ liệu được lưu trữ trên nhiều nút (site), có thể phân bố địa lý khác nhau, nhưng đối với người dùng và ứng dụng nó vẫn được nhìn như một CSDL thống nhất.

Lợi ích chính:

- Tăng khả năng mở rộng và phân bố tải.
- Tăng khả năng sẵn sàng và chịu lỗi cục bộ.
- Gắn dữ liệu với nơi phát sinh nghiệp vụ (theo chi nhánh, khu vực).

Thách thức chính:

- Đảm bảo nhất quán dữ liệu giữa các site.
- Đồng bộ và xử lý xung đột.
- Quản lý giao dịch phân tán và cơ chế recovery.

2.2 Phân mảnh dữ liệu

2.2.1 Horizontal Fragmentation

Là chia bảng theo dòng dựa trên điều kiện (predicate). Mỗi site giữ một tập dòng.

Ví dụ:

- Site A giữ tài khoản có account_number thuộc nhóm A.

- Site B giữ tài khoản có `account_number` thuộc nhóm B.

2.2.2 Vertical Fragmentation

Là chia bảng theo cột. Mỗi site giữ một nhóm cột phù hợp nghiệp vụ.

Ví dụ:

- Site 1 giữ cột nhận dạng và giao dịch.
- Site 2 giữ cột thông tin mở rộng/phân tích.

2.2.3 Hybrid Fragmentation

Là kết hợp horizontal và vertical fragmentation để đạt hiệu quả cao hơn theo từng nhóm nghiệp vụ.

2.3 Replication

2.3.1 Snapshot Replication

Sao chép dữ liệu theo từng đợt, phù hợp dữ liệu ít thay đổi hoặc chấp nhận độ trễ cập nhật.

2.3.2 Transactional Replication

Sao chép theo luồng giao dịch gần-thời-gian-thực, phù hợp hệ thống cần độ trễ thấp và nhất quán cao hơn.

Nhận xét đối với đề tài: Trong project hiện tại, trọng tâm là atomicity của giao dịch chuyển tiền, do đó 2PC được ưu tiên. Replication cấp CSDL chưa là tâm điểm triển khai.

2.4 Giao dịch phân tán

2.4.1 ACID

- Atomicity: tất cả hoặc không gì cả.
- Consistency: dữ liệu luôn hợp lệ sau giao dịch.
- Isolation: giao dịch đồng thời không làm hỏng nhau.
- Durability: đã commit thì phải bền vững.

2.4.2 Two-Phase Commit (2PC)

2PC có 2 pha:

1. Phase 1 - Prepare: coordinator yêu cầu tất cả participant chuẩn bị, participant trả lời YES/NO.
2. Phase 2 - Commit/Abort: nếu tất cả YES thì commit, ngược lại rollback.

Trong đề tài, 2PC được triển khai thông qua XA command của MySQL (`XA START`, `XA END`, `XA PREPARE`, `XA COMMIT`, `XA ROLLBACK`).

Chương 3

Phân tích hệ thống

3.1 Phát biểu bài toán

Trong hệ thống ngân hàng hiện đại, các chi nhánh thường được triển khai dưới dạng các hệ cơ sở dữ liệu độc lập (distributed sites), mỗi site quản lý dữ liệu của một tập khách hàng riêng. Khi phát sinh nhu cầu chuyển tiền giữa các tài khoản thuộc hai chi nhánh khác nhau, hệ thống cần thực hiện một giao dịch phân tán (distributed transaction) liên quan đến nhiều site.

Cụ thể, xét bài toán chuyển tiền từ tài khoản thuộc ngân hàng A (Site A) sang tài khoản thuộc ngân hàng B (Site B). Giao dịch này bao gồm hai thao tác chính:

- Trừ tiền từ tài khoản nguồn tại Site A.
- Cộng tiền vào tài khoản đích tại Site B.

Do hai thao tác này diễn ra trên hai hệ thống cơ sở dữ liệu khác nhau, hệ thống phải đảm bảo các yêu cầu nghiêm ngặt về tính toàn vẹn và nhất quán dữ liệu.

Các yêu cầu đặt ra cho hệ thống bao gồm:

- **Tính nguyên tử (Atomicity):** Giao dịch phải được thực hiện toàn bộ hoặc không thực hiện gì cả. Không được xảy ra tình trạng tiền bị trừ ở Site A nhưng không được cộng ở Site B, hoặc ngược lại.
- **Tính nhất quán (Consistency):** Sau khi giao dịch hoàn tất, dữ liệu tại các site phải luôn ở trạng thái hợp lệ, không vi phạm các ràng buộc (ví dụ: số dư tài khoản không được âm).
- **Khả năng chịu lỗi (Fault Tolerance):** Hệ thống phải có khả năng xử lý các tình huống lỗi như mất kết nối mạng, timeout, hoặc một site bị crash trong quá

trình thực hiện giao dịch. Trong các trường hợp này, hệ thống phải có cơ chế rollback hoặc phục hồi phù hợp để đảm bảo dữ liệu không bị sai lệch.

- **Theo dõi và truy vết giao dịch:** Hệ thống cần lưu lại trạng thái của giao dịch theo từng giai đoạn (phase), bao gồm các bước như chuẩn bị (prepare), cam kết (commit) hoặc hủy (abort), nhằm phục vụ cho việc kiểm tra, debug và phục hồi khi có sự cố.
- **Tính đồng bộ giữa các site:** Các site tham gia giao dịch cần phối hợp chặt chẽ thông qua một cơ chế điều phối (coordinator) để đảm bảo tất cả cùng đạt trạng thái nhất quán cuối cùng.

Để giải quyết bài toán trên, đề tài áp dụng giao thức Two-Phase Commit (2PC), trong đó một site đóng vai trò điều phối (coordinator) và các site còn lại đóng vai trò tham gia (participants). Giao thức này giúp đảm bảo rằng tất cả các site hoặc cùng commit giao dịch, hoặc cùng rollback, từ đó duy trì tính toàn vẹn dữ liệu trong môi trường phân tán.

3.2 Mô hình nghiệp vụ tổng quát

1. Người dùng đăng nhập.
2. Nhập thông tin chuyển tiền.
3. Hệ thống tìm tài khoản đích.
4. Coordinator thực hiện 2PC.
5. Trả kết quả, lưu log phase và hóa đơn giao dịch.
6. Khi có lỗi/treo, hệ thống có recovery tự động hoặc thủ công.

3.3 Use case

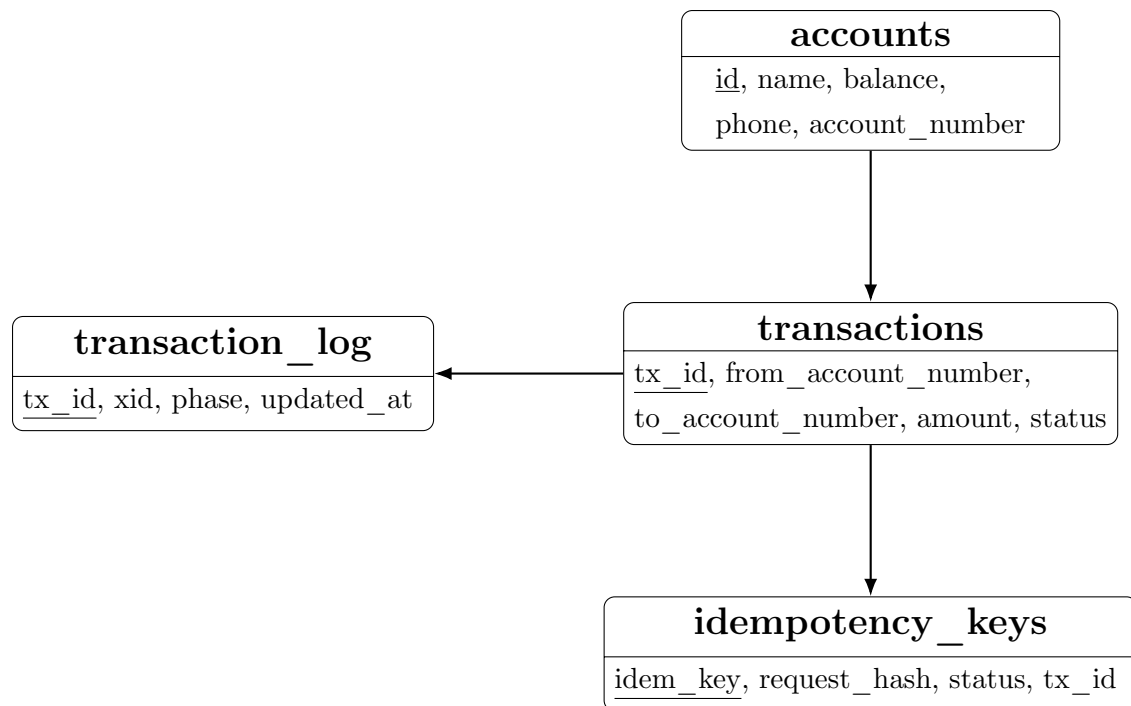
3.3.1 Tác nhân

- Người dùng (khách hàng).
- Hệ thống coordinator (backend Flask).
- Site ngân hàng A/B (MySQL participant).

3.3.2 Danh sách use case

- Đăng nhập.
- Tra cứu tài khoản.
- Chuyển tiền 2PC.
- Tra cứu trạng thái giao dịch theo tx_id.
- Xem danh sách giao dịch gần nhất.
- Chạy recovery thủ công.

article [utf8]inputenc [vietnamese]babel tikz



3.4 Lược đồ quan hệ

- accounts (id PK, name, balance, phone, password, account_number, account_type)
- transactions (id PK, tx_id UNIQUE, from_account_number, to_account_number, amount, description, status, created_at)
- transaction_log (id PK, tx_id UNIQUE, xid, from_account_number, to_account_number, amount, phase, created_at, updated_at)
- idempotency_keys (id PK, idem_key UNIQUE, request_hash, status, tx_id, http_status, response_json, timestamps)

Chương 4

Thiết kế CSDL phân tán

4.1 Phân tích Sites

Hệ thống được phân thành 3 site CSDL và 1 site ứng dụng điều phối:

Site	Port	Database	Vai trò
Site A	3306	bank1	Participant nguồn, lưu log giao dịch 2PC (transaction_log), lưu idempotency_keys
Site B	3307	bank2	Participant đích, xử lý cộng tiền
Site C	3308	bank3	Site mở rộng cho kịch bản tăng trưởng
Coordinator	5000	Flask app	Điều phối 2PC, API, recovery, monitor

4.2 Chiến lược phân mảnh

4.2.1 Horizontal fragmentation

Bảng accounts được phân bổ theo site ngân hàng. Điều kiện phân mảnh nghiệp vụ có thể biểu diễn:

- Fragment F_A : tài khoản thuộc Bank A (lưu tại bank1).
- Fragment F_B : tài khoản thuộc Bank B (lưu tại bank2).
- Fragment F_C : tài khoản thuộc Bank C (lưu tại bank3).

Predicate ví dụ (mức demo):

- `account_number` bắt đầu bằng nhóm số của từng chi nhánh/ngân hàng.
- Hoặc ánh xạ theo bảng config site thay vì hard-code predicate trong SQL.

4.2.2 Vertical fragmentation (có chủ đích)

Không chia cột theo nghĩa truyền thống trên cùng một bảng, nhưng về kiến trúc:

- Dữ liệu vận hành giao dịch tập trung log ở bank1: `transaction_log`, `idempotency_keys`.
- Dữ liệu tài khoản gốc nằm tại từng site participant.

4.2.3 Hybrid fragmentation

Hệ thống hiện tại có thể xem là hybrid practical:

- Horizontal cho dữ liệu tài khoản theo site.
- Tách lớp metadata giao dịch (log/idempotency) về site coordinator.

4.3 Giải thích lựa chọn thiết kế

4.3.1 Tại sao phân theo chi nhánh/site ngân hàng?

- Phân bổ đúng theo ownership dữ liệu (mỗi ngân hàng sở hữu số dư tài khoản của mình).
- Giảm độ phụ thuộc chéo và giảm tải cho một CSDL tập trung.
- Gần với bài toán thực tế liên ngân hàng.

4.3.2 Tại sao không replicate full toàn bộ accounts?

- Replicate full làm tăng chi phí đồng bộ và rủi ro conflict.
- Bài toán chính cần đảm bảo atomicity giao dịch, không phải sao chép toàn bộ dữ liệu read-heavy.
- 2PC + log/recovery đã đáp ứng mục tiêu môn học rõ ràng hơn replication toàn phần.

4.4 Sơ đồ phân tán (site giữ bảng nào)

Site	Bảng dữ liệu
bank1	accounts, transactions, transaction_log, idempotency_keys
bank2	accounts
bank3	accounts (mở rộng)

4.5 Chiến lược đồng bộ dữ liệu

4.5.1 Đồng bộ nghiệp vụ tiền tệ

- Đồng bộ đồng bộ (synchronous) qua 2PC.
- Chỉ commit khi tất cả participant prepare thành công.

4.5.2 Xử lý conflict

- Conflict write được giảm nhờ lock cấp transaction XA.
- Conflict yêu cầu lặp lại (double submit) được xử lý bằng idempotency_keys.
- Trường hợp lệch pha COMMIT_A được xử lý bằng compensation/recovery.

Chương 5

Cài đặt và cấu hình

5.1 Công nghệ sử dụng

- Python 3 + Flask + flask-cors.
- MySQL (InnoDB, XA transaction).
- Docker Compose để dựng nhanh 3 site CSDL + Toxiproxy.
- Pytest cho test tự động, GitHub Actions cho CI.

5.2 Cấu hình kết nối phân tán

5.2.1 Database

- bank1: localhost:3306
- bank2: localhost:3307
- bank3: localhost:3308

5.2.2 Coordinator backend

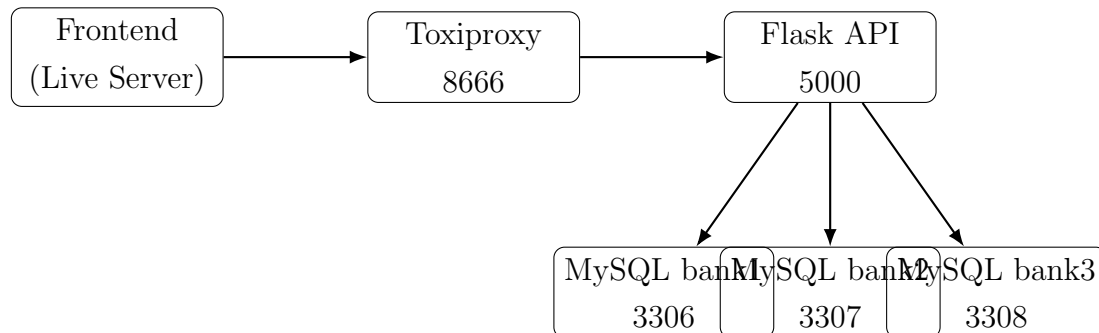
- Flask app chạy cổng 5000.
- Route chính: /api/login, /api/accounts, /api/lookup-account, /api/transfer, /api/transfer/status, /api/transfer/recent, /api/recover.

5.2.3 Toxiproxy để mô phỏng lỗi mạng

- Admin API: 8474

- Proxy service: 8666
- Mapping: 8666 -> 5000

5.3 Sơ đồ kết nối giữa các server



5.4 Liên hệ với template SQL Server/Linked Server/Replica

Form môn học đề xuất SQL Server + Linked Server + Replication. Tuy nhiên project này triển khai tương đương trên MySQL + XA:

- Linked Server được thay bằng coordinator app-level (Flask) + nhiều kết nối DB.
- Distributed transaction được thực hiện bằng XA command và 2PC.
- Replication không phải mục tiêu bắt buộc của đề tài này; thay vào đó tập trung atomicity + recovery.

Chương 6

Xây dựng và Demo

6.1 Store Procedure phân tán (tương đương trong project hiện tại)

Project hiện tại không viết SP SQL thuần cho toàn bộ luồng 2PC mà triển khai ở tầng ứng dụng (`execute_transfer()` trong backend). Về mặt thuật toán, nó tương đương một stored-procedure phân tán:

Listing 6.1: Giả mã xử lý giao dịch 2PC cấp ứng dụng

```
begin transfer_2pc(from_acc, to_acc, amount, description)
    create xid, tx_id
    log_phase(PREPARING)

    parallel:
        prepare participant A
        prepare participant B
    wait with timeout

    if any participant fail or timeout then
        rollback all
        log_phase(ABORTED or TIMEOUT)
        return error
    end if

    log_phase(PREPARED)
    log_phase(COMMITTING)

    commit A
```

```

    log_phase(COMMIT_A)

    commit B
    log_phase(COMMITTED)
    save receipt
    return success

on phase-2 partial failure:
    rollback participant chua commit
    do compensation cho participant da commit
    return error with partial_failure
end

```

6.2 Distributed Transaction 2PC

6.2.1 Lệnh XA chính được sử dụng

Listing 6.2: Tập lệnh XA trong MySQL

```

XA START 'xid';
UPDATE accounts SET balance = balance - :amount WHERE id = :
    from_id;
XA END 'xid';
XA PREPARE 'xid';

XA START 'xid';
UPDATE accounts SET balance = balance + :amount WHERE id = :
    to_id;
XA END 'xid';
XA PREPARE 'xid';

XA COMMIT 'xid';
-- or XA ROLLBACK 'xid';

```

6.2.2 Các endpoint demo chính

- POST /api/transfer
- GET /api/transfer/status/{tx_id}
- GET /api/transfer/recent

- POST /api/recover

6.3 Test case

6.3.1 Bảng test case tổng hợp

Case	Mô tả	Kết quả kỳ vọng
TC01	Happy path 2PC	COMMITTED, A trừ tiền, B cộng tiền
TC02	Bank A fail ở prepare	ABORTED, rollback toàn bộ
TC03	Bank B trả NO ở prepare	ABORTED, rollback toàn bộ
TC04	Commit A OK, commit B fail	partial_failure=true, compensation được kích hoạt
TC05	Coordinator crash sau COMMIT_A	Recovery commit tiếp B, action COMMIT_B_COMPLETED
TC06	Crash trước commit	Recovery -> ABORTED
TC07	Crash sau PREPARED	Recovery -> COMMITTED
TC08	In-doubt ở COMMITTING	Recovery -> COMMITTED
TC09	Commit gửi nhiều lần	Không nhân đôi tiền, COMMITTED
TC10	Rollback gửi nhiều lần	Hệ thống không crash, ABORTED
TC11	2 giao dịch đồng thời	Mỗi request có tx_id riêng, hệ thống ổn định
TC12	Amount <= 0	Validation error
TC13	Tài khoản đích không tồn tại	Validation error, không commit
TC14	Double submit cùng Idempotency-Key	Request sau replay, idempotent_replay=true, không tạo tx mới
TC15	Audit phase log	Đầy đủ PREPARING->PREPARED->COMMITTING->COMMIT_A->COMMITTED
TC16	Recovery từ log	Xử lý đúng theo phase pending

6.3.2 Kết quả tự động hóa kiểm thử

- Non-E2E suite: 128 passed (marker not e2e).
- Có bộ test E2E qua Toxiproxy (opt-in qua biến RUN_TOXIPROXY_E2E=1).
- Có smoke test frontend monitor/recovery.

6.3.3 Node chết / network delay / rollback scenario

Trong đề tài đã mô phỏng được các trường hợp ăn điểm:

- Node chết: TC05/TC06/TC07/TC08.
- Network delay: mô phỏng bằng toxiproxy latency/timeout.
- Kết quả: hệ thống commit đúng, rollback đúng, hoặc recovery đúng theo phase.

6.4 Giao diện (screenshot)

Do file báo cáo này không đính kèm ảnh, có thể chèn screenshot khi xuất PDF:

- Màn hình đăng nhập.
- Dashboard và monitor giao dịch 2PC.
- Màn hình kết quả transfer và tra cứu tx_id.
- Kết quả bấm recovery thủ công.

Đặt screenshot giao diện tại đây (đăng nhập / dashboard / monitor)
--

Hình 6.1: Vị trí chèn screenshot giao diện demo

Chương 7

Đánh giá hệ thống

7.1 Ưu điểm

- Triển khai được full flow 2PC trên nhiều site CSDL.
- Có xử lý trường hợp partial commit bằng compensation.
- Có cơ chế recovery từ transaction_log khi khởi động hoặc gọi API thủ công.
- Có idempotency để chống double submit trong API transfer.
- Có monitor UI để theo dõi phase theo tx_id và danh sách recent.
- Có CI tự động hóa test core + e2e qua toxiproxy.

7.2 Nhược điểm

- Chưa có cơ chế leader election cho coordinator.
- Chưa có replication cấp CSDL để đảm bảo HA đầy đủ.
- Chưa có test stress dài hạn và benchmark hiệu năng chi tiết.
- Bảo mật đang mức demo (chưa JWT, chưa mã hóa dữ liệu nhạy cảm).

7.3 So sánh với hệ tập trung

Tiêu chí	Hệ tập trung	Hệ phân tán 2PC (đề tài)
Độ phức tạp triển khai	Thấp	Cao hơn (coordinator + participant + recovery)
Atomicity liên site	Không cần xử lý liên site	Được đảm bảo bằng 2PC
Khả năng mở rộng	Hạn chế	Tốt hơn theo site
Rủi ro điểm nghẽn/cô đơn	Dễ xảy ra	Giảm theo từng site dữ liệu
Chi phí vận hành	Thấp	Cao hơn do thêm cơ chế đồng bộ/chịu lỗi

Chương 8

Kết luận và hướng phát triển

8.1 Tổng kết

Đề tài đã đạt được mục tiêu môn học CSDL phân tán:

- Triển khai bài toán chuyển tiền liên site bằng 2PC.
- Có xử lý lỗi và recovery theo phase.
- Có bộ testcase đầy đủ TC01-TC16 và tự động hóa qua CI.

8.2 Hướng phát triển

- Nâng cấp coordinator thành multi-instance (active-passive/consensus).
- Bổ sung replication và backup/restore tự động cho transaction_log.
- Tích hợp observability đầy đủ: metrics, tracing, alerting.
- Bổ sung test stress và chaos testing theo kịch bản thực tế hơn.
- Tăng cường bảo mật: bcrypt/argon2, token auth, rate limiting.

Chương 9

Tài liệu tham khảo

Tài liệu tham khảo

- [1] Tài liệu môn học Cơ sở dữ liệu phân tán, Trường Đại học Sài Gòn (SGU), 2026.
- [2] Oracle, *MySQL Reference Manual - XA Transactions*, <https://dev.mysql.com/doc/refman/8.0/en/xa.html>.
- [3] Microsoft, *Distributed Data and Transactions - Documentation*, <https://learn.microsoft.com/>.
- [4] Pallets, *Flask Documentation*, <https://flask.palletsprojects.com/>.
- [5] Tài liệu nội bộ project V-Bank 2PC: README, Architecture, Database, API, Error Handling, 2PC Protocol, truy cập trong thư mục docs của project.

Phụ lục A

Phụ lục A - Trích đoạn API quan trọng

Listing A.1: Transfer API yêu cầu idempotency

```
POST /api/transfer
Headers:
    Content-Type: application/json
    Idempotency-Key: <client-generated-key>

Body:
{
    "from_account_number": "102938475612",
    "to_account_number": "203847569801",
    "amount": 50000,
    "description": "... "
}
```

Phụ lục B

Phụ lục B - Ma trận test và CI

- Pytest marker `e2e` cho bộ test qua Toxiproxy.
- CI gồm 2 job:
 - Core tests (không e2e)
 - Toxiproxy E2E (dùng backend thật + proxy 8666)